

LESSON 6: REACT HOOKS IN DEPTH

Modern Functional Architecture, Side Effects, DOM References & Performance Optimization

1. HOOKS KI FOUNDATION: WHY HOOKS EXIST

Prior to React 16.8, React components were strictly divided into two categories:

- **Class Components:** Managed internal state, handled side effects via lifecycle methods (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`), and bound methods to this.
- **Functional Components:** Stateless, presentational units taking props and returning JSX ($UI = f(props)$).

Problems in the Class Component Model

- **Logic Duplication & Fragmentation:** Related code was split across multiple lifecycle methods. Setting up an interval or data subscription lived inside `componentDidMount`, while clearing it lived inside `componentWillUnmount`.
- **Stateful Logic Reusability:** Sharing stateful logic between classes required patterns like Higher-Order Components (HOCs) or Render Props, creating nested component hierarchies known as "Wrapper Hell."
- **this Binding Complexities:** JavaScript lexical scoping required manual function binding in class constructors or arrow function wrappers, introducing boilerplate and runtime bugs.

What are Hooks?

Hooks are built-in functions introduced in React 16.8 that allow functional components to hook into React state, lifecycle features, DOM nodes, and memory management without class syntax.

2. THE RULES OF HOOKS & INTERNAL FIBER MECHANICS

React hooks are governed by two strict rules enforced by the `eslint-plugin-react-hooks` linter:

Rule 1: Only Call Hooks at the Top Level

Do not call hooks inside loops, conditions, or nested functions. Always declare them at the very top level of your React function component before any early returns.

JavaScript

```
// ❌ CRITICAL VIOLATION: Calling hook inside a condition
```

```
function UserProfile({ userId }) {
```

```

if (!userId) {
  return <p>No user selected.</p>;
}

// If userId is missing, this hook will be skipped, corrupting the call order!
const [profile, setProfile] = useState(null);
// ...
}

// ✅ CORRECT: Always call hooks unconditionally at the top level
function UserProfile({ userId }) {
  const [profile, setProfile] = useState(null);

  if (!userId) {
    return <p>No user selected.</p>;
  }
  // ...
}

```

Rule 2: Only Call Hooks from React Functions

Call hooks only inside:

- React functional components.
- Custom Hooks (functions starting with the prefix use).

Do not invoke hooks inside regular vanilla JavaScript helper functions or class methods.

Why Call Order Matters: The Fiber Linked List

React does not track hooks using a key-value dictionary or property names. Under the hood, each React component's Fiber node maintains a single-direction **Linked List** of hook objects.

Fiber Node -> Hook 1 (useState) -> Hook 2 (useEffect) -> Hook 3 (useRef) -> null

- On the initial mount, React allocates each hook node in the linked list sequentially.
- On subsequent renders, React steps through the existing linked list in the exact order of execution to extract the cached state or memoized reference.
- If a hook is placed inside a conditional block or loop, the call index shifts, causing React to mismatch state values (e.g., assigning a boolean flag's state to a text input's state pointer), corrupting internal component state.

3. useEffect: THE SIDE EFFECT ORCHESTRATOR

Side effects occur whenever a component interacts with the outside world beyond its local render scope. Common examples include:

- Fetching data over HTTP.
- Mutating the browser DOM directly (document.title).
- Subscribing to external event emitters, WebSockets, or global window listeners.
- Setting timers (setTimeout, setInterval).

Signature

JavaScript

```
useEffect(() => {
  // Setup logic: Executes after paint
  return () => {
    // Cleanup logic: Executes prior to re-run and on unmount
  };
}, [dependencies]);
```

The Dependency Array Matrix

Dependency Array Syntax	Execution Timing	Lifecycle Analogy
Omitted	Runs after the initial render and after every subsequent render.	componentDidMount + componentDidUpdate

Dependency Array Syntax	Execution Timing	Lifecycle Analogy
<code>useEffect(fn)</code>		
Empty Array [] <code>useEffect(fn, [])</code>	Runs only once after the initial DOM paint.	<code>componentDidMount</code>
Populated Array <code>useEffect(fn, [val1, val2])</code>	Runs after the initial render and re-runs only if val1 or val2 values change between renders.	Granular <code>componentDidUpdate</code>

4. THE CLEANUP FUNCTION & MEMORY LEAK PREVENTION

Effects that subscribe to ongoing streams, setup intervals, or attach window listeners can introduce memory leaks if abandoned. To handle this, `useEffect` accepts a **cleanup function** as its return value.

When Does the Cleanup Function Execute?

1. **Before the effect re-runs** due to a dependency change (clearing the previous run's resources).
2. **When the component unmounts** completely from the DOM.

JavaScript

```
import { useState, useEffect } from "react";
```

```
function WindowResizeTracker() {
```

```

const [windowWidth, setWindowWidth] = useState(window.innerWidth);

useEffect(() => {
  const handleResize = () => setWindowWidth(window.innerWidth);

  // 1. SETUP: Attach global browser listener
  window.addEventListener("resize", handleResize);

  // 2. CLEANUP: Detach listener to prevent memory leaks and zombie references
  return () => {
    window.removeEventListener("resize", handleResize);
  };
}, []); // Empty array ensures listener is bound once on mount

return <p>Window Width: {windowWidth}px</p>;
}

```

5. ROBUST DATA FETCHING & RACE CONDITIONS WITH `useEffect`

Data fetching within `useEffect` must handle asynchronous race conditions, loading states, and error propagation.

The Network Race Condition Trap

If a user changes filters rapidly (e.g., from query "A" to "B" to "C"), three asynchronous network calls are dispatched. If request "A" finishes after request "C" due to network latency, older, stale data will overwrite the newer result.

Production Pattern: Race Condition Prevention via Cancellation Flags

JavaScript

```
import { useState, useEffect } from "react";
```

```
function PostSearch({ postId }) {  
  const [post, setPost] = useState(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);  
  
  useEffect(() => {  
    let isCurrent = true; // Boolean cancellation flag  
    setLoading(true);  
    setError(null);  
  
    fetch(`https://jsonplaceholder.typicode.com/posts/${postId}`)  
      .then((res) => {  
        if (!res.ok) throw new Error(`HTTP Error status: ${res.status}`);  
        return res.json();  
      })  
      .then((data) => {  
        // Prevent setting state if another request initiated before this resolved  
        if (isCurrent) {  
          setPost(data);  
          setLoading(false);  
        }  
      })  
      .catch((err) => {  
        if (isCurrent) {  
          setError(err.message);  
          setLoading(false);  
        }  
      })  
  });  
}
```

```

    }
  });

  // Cleanup: Invalidate this request thread if postId updates or component unmounts
  return () => {
    isCurrent = false;
  };
}, [postId]); // Re-runs whenever target postId changes

if (loading) return <p>Loading resource...</p>;
if (error) return <p style={{ color: "red" }}>Failed: {error}</p>;
if (!post) return null;

return (
  <article>
    <h3>{post.title}</h3>
    <p>{post.body}</p>
  </article>
);
}

```

6. useRef: DOM ACCESS & MUTABLE REFERENCES

The useRef hook returns a mutable object with a single property: { current: initialValue }.

Key Characteristics

- **Persistence:** The returned object persists across all render cycles for the lifetime of the component instance.
- **No Re-rendering on Mutation:** Unlike useState, mutating .current directly does **not** trigger a component re-render.

- **Synchronous Updates:** Mutations to `.current` are applied synchronously.

`useState`: Updating state -> Triggers Component Re-render -> UI Syncs

`useRef`: Updating `.current` -> NO Re-render -> Memory pointer updates silently

Common Use Cases

1. Direct DOM Element Access

JavaScript

```
import { useRef, useEffect } from "react";
```

```
function AutoFocusSearch() {
```

```
  const searchInputRef = useRef(null);
```

```
  useEffect(() => {
```

```
    // Access native imperative DOM API directly
```

```
    searchInputRef.current.focus();
```

```
  }, []);
```

```
  return <input ref={searchInputRef} placeholder="Search query..." />;
```

```
}
```

2. Storing Instance Variables (Timers, Previous State)

JavaScript

```
import { useState, useRef } from "react";
```

```
function Stopwatch() {
```

```
  const [seconds, setSeconds] = useState(0);
```

```
  const timerIdRef = useRef(null); // Keeps track of interval ID across renders
```



```

const startTimer = () => {
  if (timerIdRef.current !== null) return;
  timerIdRef.current = setInterval(() => {
    setSeconds((prev) => prev + 1);
  }, 1000);
};

const stopTimer = () => {
  clearInterval(timerIdRef.current);
  timerIdRef.current = null; // Reset pointer without forcing re-render
};

return (
  <div>
    <h3>Elapsed Time: {seconds}s</h3>
    <button onClick={startTimer}>Start</button>
    <button onClick={stopTimer}>Stop</button>
  </div>
);
}

```

7. useMemo: EXPENSIVE COMPUTATION OPTIMIZATION

Every time a component re-renders, all statements in the function body re-execute from top to bottom. If a calculation performs heavy CPU work (filtering large datasets, parsing documents), running it on every render degrades performance.

useMemo caches the **result** of a calculation:

JavaScript

```
const memoizedValue = useMemo(() => {
```

```
    return executeHeavyCalculation(a, b);
  }, [a, b]);
```

- Runs during rendering.
- Recomputes only when dependencies in the array change.
- If dependencies are unchanged, it reuses the previously calculated value.

JavaScript

```
import { useState, useMemo } from "react";
```

```
function DataAnalyticsView({ dataset }) {
  const [filterQuery, setFilterQuery] = useState("");
  const [darkMode, setDarkMode] = useState(false);

  // Expensive processing: Only run when dataset or filter changes, NOT on theme toggles
  const processedMetrics = useMemo(() => {
    console.log("Analyzing data...");
    return dataset
      .filter((item) => item.name.toLowerCase().includes(filterQuery.toLowerCase()))
      .reduce((acc, curr) => acc + curr.score, 0);
  }, [dataset, filterQuery]);

  return (
    <div style={{ background: darkMode ? "#121212" : "#FFF" }}>
      <button onClick={() => setDarkMode(!darkMode)}>Toggle Visual Theme</button>
      <input
        value={filterQuery}
        onChange={(e) => setFilterQuery(e.target.value)}
      />
    </div>
  );
}
```

```

    placeholder="Filter points..."

  />

  <p>Calculated Aggregate Score: {processedMetrics}</p>

</div>

);
}

```

8. useCallback: STABILIZING FUNCTION REFERENCES

In JavaScript, functions are first-class objects. Every time a component renders, every function declared within its body is recreated with a **new memory reference**:

JavaScript

```

const fn1 = () => {};

const fn2 = () => {};

fn1 === fn2; // false (different references in memory)

```

If you pass an inline function to a child component wrapped in `React.memo`, the child will detect a new reference for that prop and re-render anyway, bypassing the memoization check.

`useCallback` caches the **function definition** across renders:

JavaScript

```

const memoizedCallback = useCallback((arg) => {
  doSomething(arg, dependency);
}, [dependency]);

```

Architectural Comparison: `useMemo` vs `useCallback`

- `useMemo(() => fn, [deps])` memoizes the **return value** of the function.
- `useCallback(fn, [deps])` memoizes the **function instance itself**.
- In fact, `useCallback(fn, deps)` is syntactic sugar for:

JavaScript

```
useMemo(() => fn, deps)
```

JavaScript

```

import React, { useState, useCallback } from "react";

// Child wrapped in React.memo to prevent unnecessary renders
const ActionButton = React.memo(({ onAction, label }) => {
  console.log(`Rendered button: ${label}`);
  return <button onClick={onAction}>{label}</button>;
});

function CounterDashboard() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("");

  // Preserves identical memory reference across re-renders
  const handleReset = useCallback(() => {
    setCount(0);
  }, []); // Empty deps: Reference stays stable forever

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <p>Count: {count}</p>
      <button onClick={() => setCount((c) => c + 1)}>Increment</button>
      { /* ActionButton will NOT re-render when 'text' changes */ }
      <ActionButton onAction={handleReset} label="Reset Metric" />
    </div>
  );
}

```

```
}
```

9. ENTERPRISE INTEGRATION: FULL HOOK ECOSYSTEM

Below is a complete implementation showing how `useState`, `useEffect`, `useRef`, `useMemo`, and `useCallback` coordinate in a production search component:

JavaScript

```
import React, { useState, useEffect, useRef, useMemo, useCallback } from "react";
```

```
// Child list component memoized to avoid re-rendering on unrelated input changes
```

```
const ResultList = React.memo(({ items, onSelectItem }) => {  
  console.log("[Render] ResultList component executed");  
  return (  
    <ul>  
      {items.map((item) => (  
        <li key={item.id} style={{ margin: "6px 0" }}>  
          <span>{item.title}</span>  
          <button onClick={() => onSelectItem(item.id)} style={{ marginLeft: 8 }}>  
            Inspect  
          </button>  
        </li>  
      ))}  
    </ul>  
  );  
});
```

```
export default function SearchEngineMaster() {  
  const [data, setData] = useState([]);
```

```
const [search, setSearch] = useState("");
const [selectedId, setSelectedId] = useState(null);
const [darkTheme, setDarkTheme] = useState(false);

// 1. useRef: Retain reference to real DOM input for programmatic focus
const searchInputRef = useRef(null);

// 2. useEffect: Initial setup to fetch mock data and focus input
useEffect(() => {
  let active = true;

  // Focus input on initial mount
  if (searchInputRef.current) {
    searchInputRef.current.focus();
  }

  // Fetch initial dataset
  fetch("https://jsonplaceholder.typicode.com/posts?_limit=8")
    .then((res) => res.json())
    .then((records) => {
      if (active) setData(records);
    })
    .catch((err) => console.error("Initial load failed:", err));

  return () => {
    active = false;
  };
}, []);
```

```

    };
  }, []);

// 3. useMemo: Cache filtered dataset to prevent recalculating on theme toggles
const filteredRecords = useMemo(() => {
  console.log("[Computing] Filtering record dataset...");
  return data.filter((item) =>
    item.title.toLowerCase().includes(search.toLowerCase())
  );
}, [data, search]);

// 4. useCallback: Stable function reference passed down to memoized child
const handleItemSelection = useCallback((id) => {
  console.log(`Item selected: ${id}`);
  setSelectedId(id);
}, []);

return (
  <div style={{
    padding: 24,
    background: darkTheme ? "#1E293B" : "#F8FAFC",
    color: darkTheme ? "#F8FAFC" : "#0F172A",
    minHeight: "100vh"
  }}>
    <h2>Hook Ecosystem Control Panel</h2>

```

```

<div style={{ marginBottom: 16 }}>
  <button onClick={() => setDarkTheme((prev) => !prev)}>
    Toggle Mode ({darkTheme ? "Dark" : "Light"})
  </button>
</div>

```

```

<div style={{ marginBottom: 16 }}>
  <input
    ref={searchInputRef}
    value={search}
    onChange={(e) => setSearch(e.target.value)}
    placeholder="Filter records..."
    style={{ padding: 8, width: 300 }}
  />
</div>

```

```

<p>Selected Record ID: <strong>{selectedId ?? "None"}</strong></p>

```

```

{/* ResultList re-renders only when filteredRecords or handleItemSelection changes */}

```

```

<ResultList items={filteredRecords} onSelectItem={handleItemSelection} />

```

```

</div>

```

```

);

```

```

}

```

10. CUSTOM HOOKS INTRODUCTION

A Custom Hook is an abstraction mechanism that extracts stateful logic out of components into reusable functions.

Design Principles

- **Naming Standard:** Must start with the prefix use (e.g., useFetch, useLocalStorage). This tells the React linter to enforce the Rules of Hooks within it.
- **Isolated State:** Custom hooks share **stateful logic**, not state itself. Each component calling a custom hook receives an independent instance of the internal state.

Production Example: Reusable useFetch Hook

JavaScript

```
import { useState, useEffect } from "react";

export function useFetch(url) {

  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {

    let isMounted = true;

    setLoading(true);
    setError(null);

    fetch(url)
      .then((res) => {
        if (!res.ok) throw new Error(`Fetch failed: ${res.statusText}`);
        return res.json();
      })
      .then((resolvedData) => {
        if (isMounted) {
          setData(resolvedData);
        }
      })
  }, [url]);
}
```

```

        setLoading(false);
    }
})
.catch((err) => {
    if (isMounted) {
        setError(err.message);
        setLoading(false);
    }
});

return () => {
    isMounted = false;
};
}, [url]);

return { data, loading, error };
}

```

Consuming the Custom Hook

JavaScript

```

function UserDirectory() {
    const { data: users, loading, error } = useFetch("https://jsonplaceholder.typicode.com/users");

    if (loading) return <p>Loading users...</p>;
    if (error) return <p style={{ color: "red" }}>Error: {error}</p>;

    return (

```

```

<ul>
  {users.map((u) => (
    <li key={u.id}>{u.name} {u.email}</li>
  ))}
</ul>

);
}

```

11. HOOK ANTI-PATTERNS & PERFORMANCE PITFALLS

1. Over-Memoizing Everything

- **Mistake:** Wrapping every primitive calculation in `useMemo` and every basic inline handler in `useCallback`.
- **Cost:** `useMemo` and `useCallback` carry memory and CPU overhead to instantiate dependency arrays and run reference checks. For lightweight operations, this costs more than re-creating the function or recalculating the value on render.
- **Rule of Thumb:** Use them when passing callbacks to memoized children (`React.memo`), when maintaining stable dependency references in other hooks (`useEffect`), or when running genuinely expensive operations ($O(n^2)$ array processing, heavy parsing).

2. Missing Dependencies in Dependency Arrays

- **Mistake:** Omitting props or state values from the dependency array to prevent an effect from re-running.
- **Cost:** Causes **stale closures**. The effect captures values from older render cycles, leading to bugs like unsynchronized state or outdated data.
- **Fix:** Use functional state updates (`setCount(c => c + 1)`) or move internal function logic directly into the effect.

3. Reading or Writing Refs During Rendering

- **Mistake:** Modifying `ref.current` during the component render phase.
- **Cost:** React assumes the render phase is a pure calculation free of side effects. Writing to refs during render leads to race conditions and erratic behavior in concurrent rendering modes.

- **Fix:** Only read or mutate refs inside event handlers or within `useEffect`.

12. TECHNICAL INTERVIEW & RECAP CHEAT SHEET

High-Yield Questions & Concise Answers

Q1: What is the underlying reason behind the "Only call hooks at the top level" rule?

Answer: React tracks hook state internally using a singly linked list on the Fiber node. Because it relies on call order to match state with the correct hook across renders, hooks cannot be placed inside conditions, loops, or nested functions where execution order could change.

Q2: What is the difference between `useEffect` and `useLayoutEffect`?

Answer: `useEffect` runs asynchronously after the browser paints changes to the screen, preventing blocking. `useLayoutEffect` runs synchronously after all DOM mutations but *before* the browser paints, which is helpful for reading layout metrics (scroll position, bounding box) and applying updates before the user sees a visual flicker.

Q3: Does updating a `useRef.current` trigger a component re-render?

Answer: No. `useRef` returns a plain JavaScript object whose `.current` property can be mutated synchronously without notifying React's reconciliation engine.

Q4: How does `useCallback` prevent child components from re-rendering?

Answer: Functions declared inside components receive a new memory reference on every render. Wrapping them in `useCallback` returns the same cached function reference as long as its dependencies are unchanged. When paired with `React.memo` on the child, this preserves shallow prop equality and skips unnecessary re-renders.

Q5: What happens if you update state inside `useEffect` without a dependency array?

Answer: You create an infinite loop. The effect runs after the render, the state update triggers another render, which re-runs the effect, repeating the cycle indefinitely.